ZEISS India Hackathon · Beyond Code 2026

AI/ML for Advanced Frugal Innovation

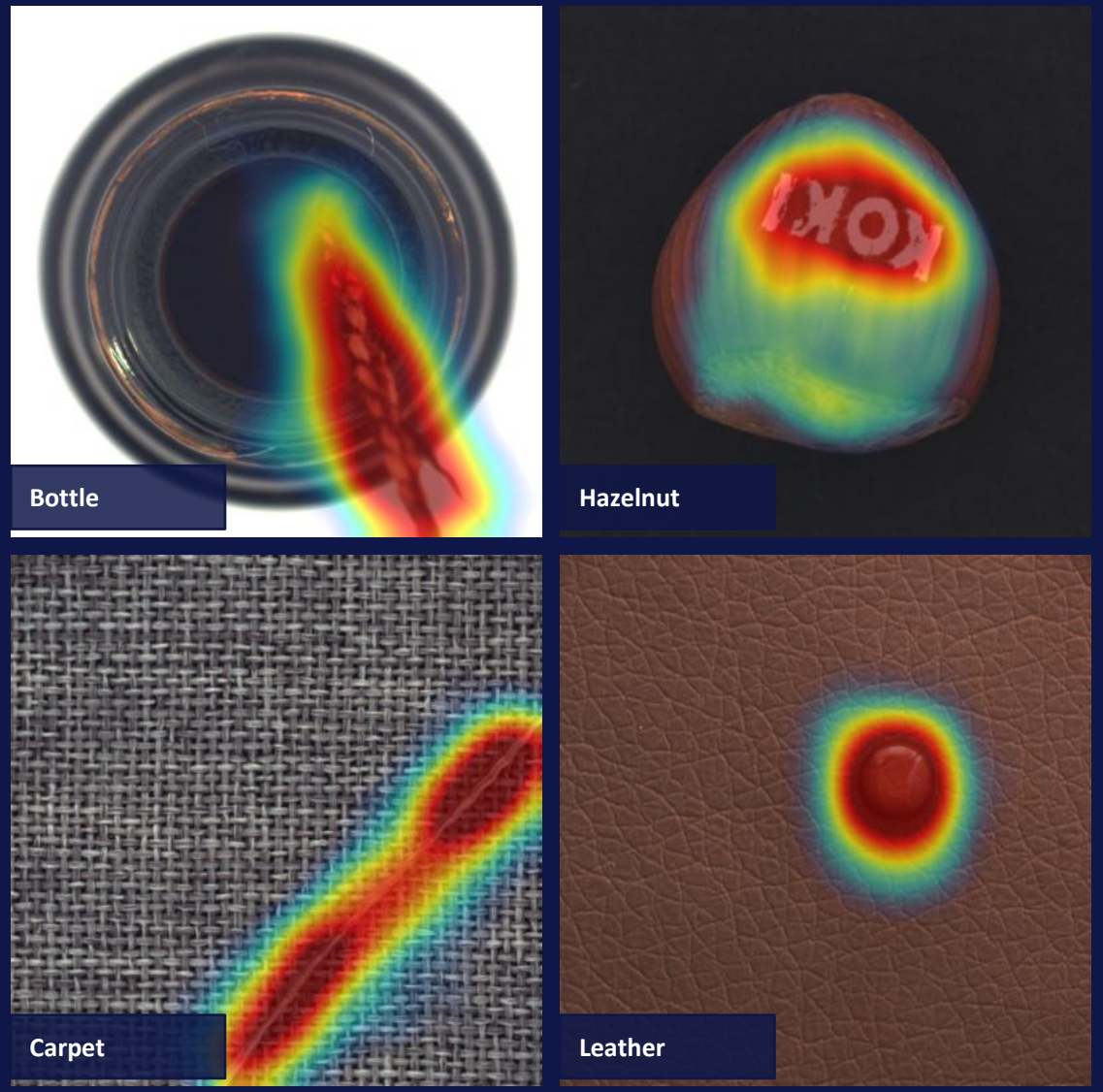
MID-PROGRESS UPDATE

NetraQC

Low-cost surface inspection for small factories, inspired by ZEISS SurfMax

Team Tensor-hive, IIT Kanpur

8 November 2026



MVTec AD test images with NetraQC heatmaps

Our objective, and what we have done so far

Objective An inspection station a small factory can set up from about 20 good parts. It flags surface defects with a heatmap, runs offline on a ₹12k Raspberry Pi kit, and uses ML to choose the cheapest design that still catches defects.

Sprint goal By 15 November: a working offline prototype, tested beyond one product type, with the lighting question measured.

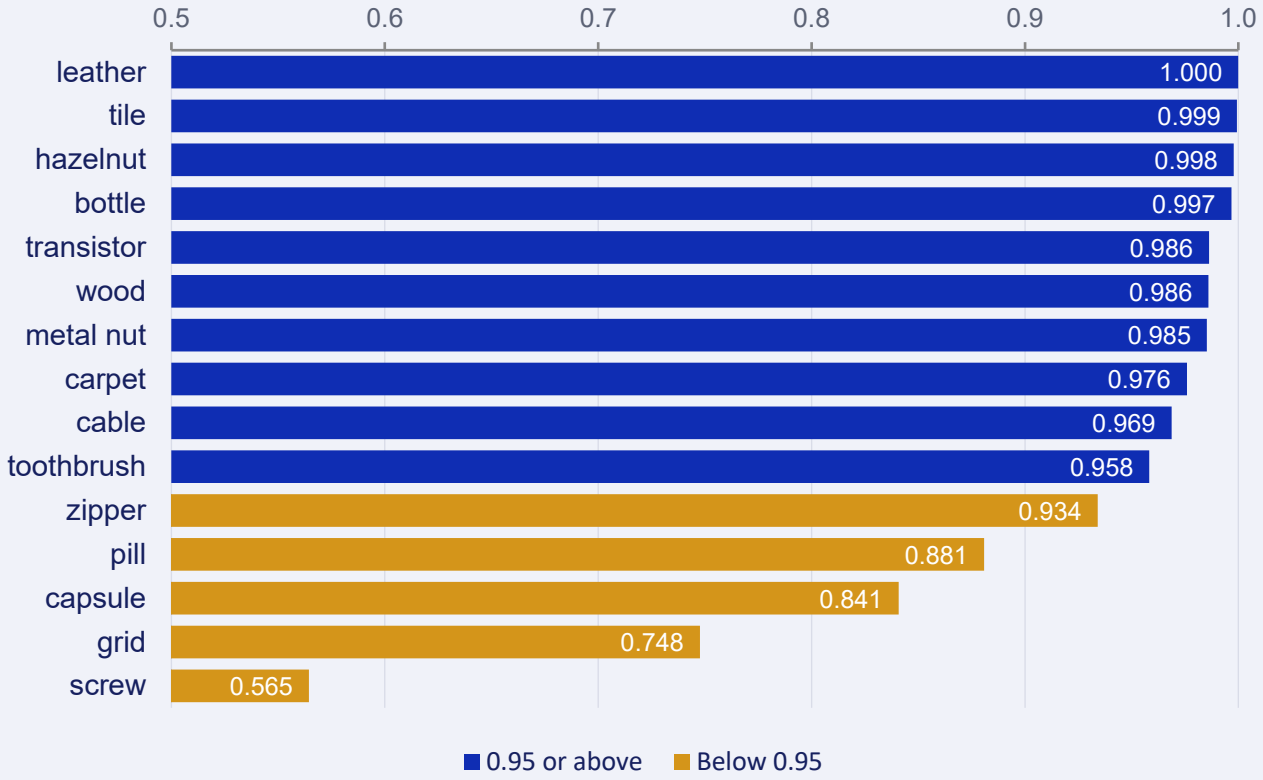
#	DONE IN THE SPRINT	STATUS	WHAT WE FOUND
1	**Compress the detector for edge devices (ONNX, INT8)**	**Done**	11.1 → 2.9 MB and 1.6× faster on 4 CPU threads; AUROC drop at most 0.004 on 4 categories
2	**Check that light adds up, so light combinations can be simulated**	**Done: no**	No: on Eyecandies renders the all-lights shot misses the sum of single lights by 28–57%. Simulation needs linear RAW captures
3	**Compare one shot with several lights**	**Early signal**	Four directional lights: AUROC 0.735 vs. 0.583 for one all-lights shot (cropped, 320 px; 50 samples)
4	**Search for the fewest lights that keep accuracy**	**Early signal**	All 63 combinations scored: 2 directional lights (0.755) beat all 6 together (0.699). Picked on the test samples, so optimistic
5	**Run on all 15 MVTec AD product types, no per-type tuning**	**Done**	Mean image AUROC 0.921; 10 of 15 at 0.95 or above; 923/1258 defects caught

Already built before the sprint: the anomaly detector (metal nut: 0.985 AUROC, 82 of 93 defects caught, 0 of 22 false alarms) and the ML design optimiser (300 designs; a 131 MB model bought no accuracy over our 13.5 MB one). Code: github.com/vatsal3512/zeiss-netraqc

One fixed design on 15 product types: mean AUROC 0.921

Same model, settings and threshold rule for every type; each set up from 20 defect-free images.

Image AUROC by product type



923 / 1258

defective images caught at the threshold

53 / 467

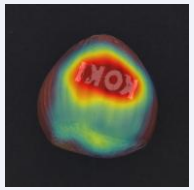
defect-free images wrongly flagged

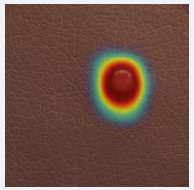
What this taught us

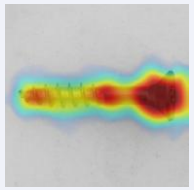
Weakest where defects are tiny or parts rotate: zipper 0.934, pill 0.881, capsule 0.841, grid 0.748, screw 0.565. Fine detail is lost at 224 px.

The threshold rule needs work: leather ranks perfectly (AUROC 1.000) yet 15 of 32 good samples were flagged. 20 images are too few to set a cut-off.

Heatmaps hold up: mean pixel AUROC 0.962, even where image scores are weak.


hazelnut


leather


screw (weak)

INT8 keeps accuracy, and directional light shows an early gain

Step 1: INT8 keeps accuracy (4 CPU threads)

PRODUCT	FP32 AUROC	INT8 AUROC	FP32 → INT8 MS
metal nut	0.985	**0.985**	21.4 → 14.3
screw	0.565	**0.568**	21.3 → 13.7
carpet	0.976	**0.972**	22.3 → 15.0
capsule	0.841	**0.838**	24.0 → 14.1

Model 11.1 MB → 2.9 MB. Calibrated only on defect-free metal-nut images, yet it holds on the other types.

Steps 2–4: lighting on Eyecandies

Light did not add up. Predicting the all-lights shot from single-light shots left 28–57% error: each render is tone-mapped. Our station must save linear RAW images to simulate light combinations.

Directional light helps (early signal): cropped to the candy at 320 px, each single directional light (0.643–0.726) beat one all-lights shot (0.583); all four together reached 0.735. An early signal for multi-angle lighting, not proof: 50 samples, 16 defective, and only 3/16 caught at the threshold. At 224 px on the full frame the detector was near chance.

Plan to the 15 November deadline

9–11 Nov	Working prototype: browser app (FastAPI) running the INT8 model offline; image in, heatmap + PASS/FAIL + log out
11–13 Nov	Fix weak types: run the design optimiser per product type (input size, feature depth); a better threshold rule
13–15 Nov	Confirm the lighting signal on more Eyecandies samples and classes; final deck, demo video and code

Our ask for the ZEISS leaders
What false-alarm rate would a small factory accept? And are there sample parts or images we could test on?